UART Receiver

Introduction:

UART (Universal Asynchronous Receiver Transmitter) is a widely used serial communication protocol for transferring data between digital systems using a single transmit line and a single receive line. The UART receiver module implemented here captures serial data from the input line, samples it according to the configured baud rate, and reconstructs it into 8-bit parallel data. This design is implemented using a finite state machine (FSM) and supports the standard UART frame format consisting of one start bit, eight data bits, and one stop bit. The receiver is designed for operation at 115200 baud with a 25 MHz system clock, requiring 217 clock cycles per bit for timing alignment and sampling accuracy.

Verilog Code Description:

The UART receiver is implemented as a synchronous FSM-based design with four operating states: IDLE, START, RECV, and STOP. The module accepts the system clock (clk), active-low reset (rst_n), and serial input (serial_in). It outputs the received 8-bit parallel byte on data_out and asserts data_ready for one clock cycle when a complete byte has been successfully received.

To ensure reliable sampling of the asynchronous serial input, the design first passes serial_in through a two-stage synchronizer (rx_meta and rx_sync). This prevents metastability when sampling the incoming UART stream.

The internal logic uses:

- state to track the current FSM state,
- clk_count to count clock cycles within each UART bit period,
- bit_index to track which data bit is being received,
- data_buf to assemble the received byte before presenting it on the output.

The receiver detects the falling edge of the start bit, waits until the center of the bit for validation, samples each incoming data bit at the configured baud interval, and then checks the stop bit before asserting data_ready.

Block Diagram:

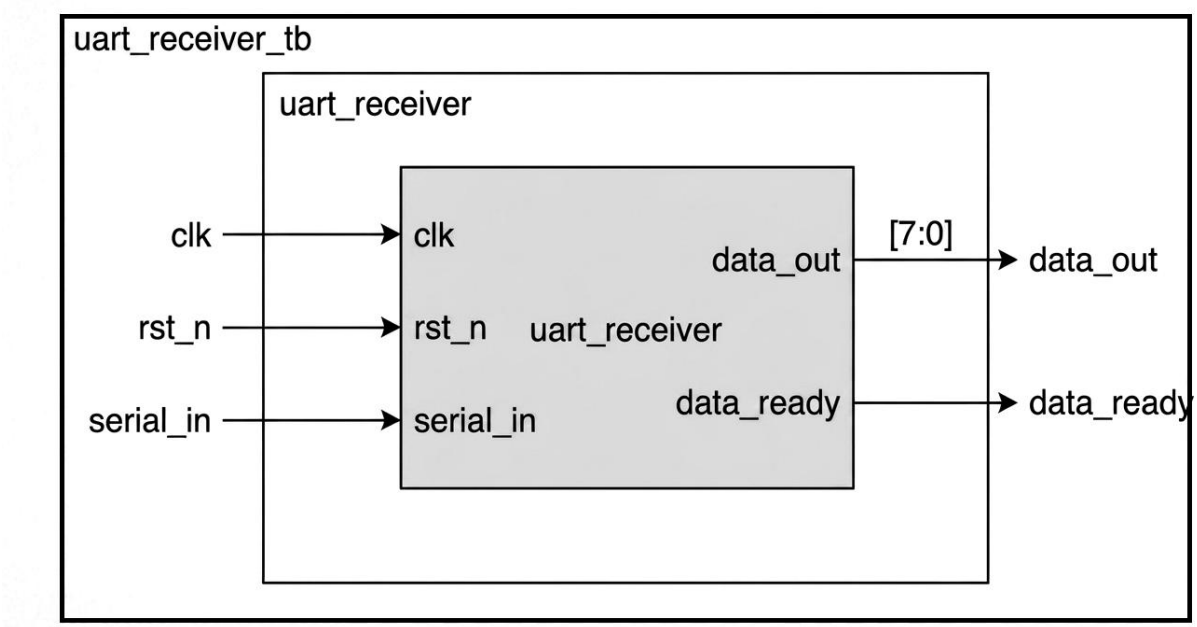


Fig: UART Receiver

Data Transmission Flow in UART:

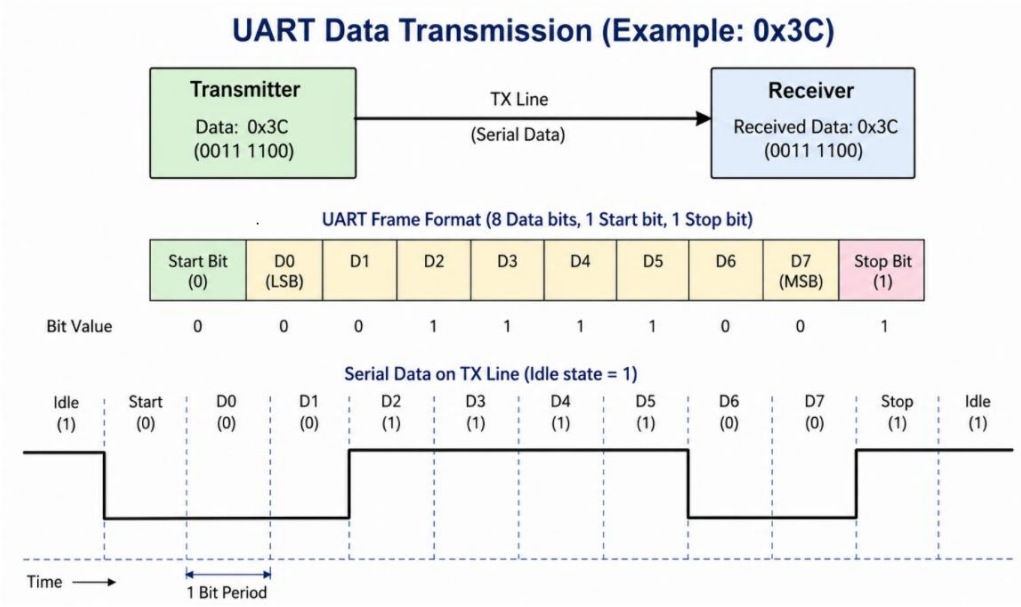


Fig: Data Transmission Flow in UART

State Machine Description:

State	Description	Next State Condition
IDLE	Waits for the serial line to go low, indicating a start bit	If serial_in goes low, transition to **START**
START	Waits for half a bit period and validates the start bit	If line is still low, transition to **RECV**; otherwise return to **IDLE**
RECV	Samples 8 data bits at each baud interval and stores them in data_buf	After all 8 bits are received, transition to **STOP**
STOP	Waits one bit period for the stop bit and completes the frame	Transfers data_buf to data_out, asserts data_ready, then returns to **IDLE**

Timing Parameters:

- **System Clock:** 25 MHz (40 ns period)
- **Baud Rate:** 115200 bps
- **Clocks Per Bit:** 217 (25,000,000 / 115,200)
- **Bit Period:** 8.68 µs (1 / 115,200)

These timing values ensure that the UART receiver samples each bit near the centre of the bit window for reliable reception

Application:

UART receivers are essential components in embedded systems, microcontroller communications, sensor interfacing, debugging systems, wireless module communications, and any application requiring reliable serial data reception with standard baud rates.

Procedure To Create Lab:

Step 1: Set Up Project Directory

1.1: Create directory for project: `mkdir uart_receiver`

1.2: Open project directory: `cd uart_receiver`

Step 2: Create Design File

2.1: Open gvim editor for design with command: `gvim uart_receiver.v`

RTL Design Code:

```
module uart_receiver
#(
    parameter integer CLKS_PER_BIT = 217
)
(
    input  wire       clk,
    input  wire       rst_n,
    input  wire       serial_in,
    output reg [7:0]  data_out,
    output reg        data_ready
);

  localparam IDLE  = 2'd0;
  localparam START = 2'd1;
  localparam RECV  = 2'd2;
  localparam STOP  = 2'd3;

  reg [1:0] state;
  reg [7:0] clk_count;
  reg [2:0] bit_index;
  reg [7:0] data_buf;

  reg rx_meta;
  reg rx_sync;

  // Synchronizer for asynchronous UART input
  always @(posedge clk) begin
    rx_meta <= serial_in;
    rx_sync <= rx_meta;
  end

  // UART Receiver FSM
  always @(posedge clk) begin
    if (!rst_n) begin
      state      <= IDLE;
      clk_count  <= 8'd0;
      bit_index  <= 3'd0;
      data_buf   <= 8'd0;
      data_out   <= 8'd0;
      data_ready <= 1'b0;
    end
```

```
  else begin
    data_ready <= 1'b0;

    case (state)

      IDLE: begin
        clk_count <= 8'd0;
        bit_index <= 3'd0;

        if (rx_sync == 1'b0)
          state <= START;
      end

      START: begin
        if (clk_count == (CLKS_PER_BIT >> 1)) begin
          clk_count <= 8'd0;

          if (rx_sync == 1'b0)
            state <= RECV;
          else
            state <= IDLE;
        end
        else begin
          clk_count <= clk_count + 1'b1;
        end
      end

      RECV: begin
        if (clk_count == CLKS_PER_BIT-1) begin
          clk_count <= 8'd0;

          // UART sends LSB first
          data_buf[bit_index] <= rx_sync;

          if (bit_index == 3'd7) begin
            bit_index <= 3'd0;
            state     <= STOP;
          end
          else begin
            bit_index <= bit_index + 1'b1;
          end
        end
        else begin
          clk_count <= clk_count + 1'b1;
```

```
        end
      end

      STOP: begin
        if (clk_count == CLKS_PER_BIT-1) begin
          clk_count  <= 8'd0;
          data_out   <= data_buf;
          data_ready <= 1'b1;
          state      <= IDLE;
        end
        else begin
          clk_count <= clk_count + 1'b1;
        end
      end

      default: begin
        state <= IDLE;
      end

    endcase
  end
end

endmodule
```

2.2: Save the file using command: `:wq!`

Step 3: Create Testbench File

3.1: Open gvim editor for testbench with command: `gvim uart_receiver_tb.v`

Testbench:

```
`timescale 1ns/1ps
module uart_receiver_tb;

  localparam CLK_PERIOD   = 40;
  localparam CLKS_PER_BIT = 217;
  localparam BIT_TIME     = CLK_PERIOD * CLKS_PER_BIT;

  reg clk;
  reg rst_n;
  reg serial_in;

  wire [7:0] data_out;
  wire       data_ready;
```

```
  integer rx_count;

  uart_receiver #(
    .CLKS_PER_BIT(CLKS_PER_BIT)
  ) dut (
    .clk(clk),
    .rst_n(rst_n),
    .serial_in(serial_in),
    .data_out(data_out),
    .data_ready(data_ready)
  );

  // Clock generation
  always #(CLK_PERIOD/2) clk = ~clk;

  // UART frame transmitter
  task send_uart_frame(input [7:0] payload);
    integer i;
    begin
      serial_in = 1'b0; #(BIT_TIME);   // Start bit

      for (i = 0; i < 8; i = i + 1) begin
        serial_in = payload[i];
        #(BIT_TIME);
      end

      serial_in = 1'b1; #(BIT_TIME);   // Stop bit
    end
  endtask

  // Monitor every received byte
  always @(posedge clk) begin
    if (data_ready) begin
      rx_count = rx_count + 1;
      $display("RX[%0d] @ %0t ns = %h", rx_count, $time,
data_out);
    end
  end

  initial begin
    clk       = 1'b0;
    rst_n     = 1'b0;
    serial_in = 1'b1;
```

```
    rx_count  = 0;

    // Reset
    #(10 * CLK_PERIOD);
    rst_n = 1'b1;
    #(5 * CLK_PERIOD);
    // Send two UART frames
    send_uart_frame(8'h3C);
    #(BIT_TIME);              // optional gap
    send_uart_frame(8'h2F);

    // Wait for second byte to complete
    #(3 * BIT_TIME);

    $display("Final data_out = %h", data_out);

    if (rx_count == 2)
      $display("PASS: Two bytes received successfully");
    else
      $display("FAIL: Expected 2 bytes, received %0d",
rx_count);
    #(5 * CLK_PERIOD);
    $finish;
  end
  initial begin
     $dumpfile("uart_receiver_tb.vcd");
    $dumpvars(0, uart_receiver_tb);
  end

endmodule
```

3.2: Save the testbench file using command: `:wq!`

3.3: Check the saved file in current directory: use command `ls`

Step 4: Simulate the Design with Verilator

a) Create a simulation script : `gvim run.sh`

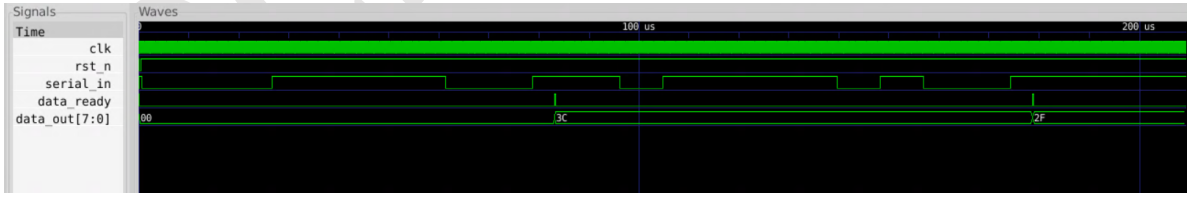
```
#!/bin/bash

# Step 1: Compile RTL and Testbench using Verilator

verilator --binary -j 0 -Wall uart_receiver.v
uart_receiver_tb.v --top uart_receiver_tb --timing --trace --
CFLAGS "-std=c++20"

# Step 2: Build directory

cd obj_dir || { echo "Error: obj_dir not found"; exit 1; }

# Step 3: Build simulation executable

make -f Vuart_receiver_tb.mk Vuart_receiver_tb || { echo
"Error: Compilation failed"; exit 1; }

# Step 4: Run simulation

./Vuart_receiver_tb || { echo "Error: Simulation failed"; exit
1; }

# Step 5: Open waveform

gtkwave uart_receiver_tb.vcd
```

b) Make the script executable: `chmod +x run.sh`

c) Run the simulation and waveform viewer with : `./run.sh`

Waveform Output:

Analyze the waveform to verify correct operation of the UART receiver



Waveform Observation:

The waveform should show the following sequence for data transmission:

- The start bit (logic 0) is received.
- The 8 data bits are sampled correctly.
- The stop bit (logic 1) is received.
- The data_ready signal is asserted when the full byte is received.

Conclusion:

The simulation output demonstrates the successful operation of the UART receiver module. The testbench transmits two hexadecimal values, 0x3C (00111100 in binary) and 0x2F (00101111 in binary), through the serial input using the standard UART frame format. For each frame, the receiver correctly detects the start bit, samples all 8 data bits at the required baud intervals, and validates the stop bit. After each successful reception, the module asserts data_ready for one clock cycle and updates data_out with the received byte.

The terminal output confirms correct reception of both bytes:

```
RX[1] @ 83220000 ns = 3c

RX[2] @ 178700000 ns = 2f

Final data_out = 2f

PASS: Two bytes received successfully
```

This verifies that the UART receiver correctly handles consecutive byte transfers and reconstructs each serial frame into valid parallel data according to the expected UART timing and protocol requirements.